

Pró-Reitoria Acadêmica
Curso de Ciência da Computação
Trabalho

ANALISADOR LÉXICO

Autores: Gabriel Baptista Gallo, Arthur Lemos Bendini

Orientador: Marcelo Eustaquio Soares de Lima Junior

BRASÍLIA-DF

2026

Gabriel Baptista Gallo, Arthur Lemos Bendini

ANALISADOR LÉXICO

Trabalho apresentado à graduação do Curso de
Ciência da Computação da Universidade Católica
de Brasília.

Orientador: Marcelo Eustaquio Soares de Lima Junior

BRASÍLIA-DF

2026

1. DESCRIÇÃO DAS ESTRUTURAS DE DADOS

1.1 Estrutura de Token

A estrutura Token é a principal unidade de informação manipulada pelo analisador léxico. Cada instância dessa estrutura representa um token reconhecido a partir do código-fonte, contendo tanto sua classificação quanto informações de localização.

```
typedef struct {  
    char token[30];  
    char lexema[100];  
    int linha;  
    int coluna;  
} Token;
```

Descrição detalhada dos campos:

Campo	Tipo	Descrição
token	char[30]	Nome do token (ex: ID, NUM_INT, KW_PROGRAM)
lexema	char[100]	Sequência de caracteres reconhecida no código-fonte
linha	int	Linha onde o token foi encontrado
coluna	int	Coluna inicial do token

O lexema é convertido para minúsculo durante a leitura para garantir que a linguagem seja tratada como case-insensitive.

A estrutura é preenchida completamente antes de ser retornada pela função `proximoToken()`.

A posição (linha e coluna) corresponde ao início do lexema.

1.2 Tabela de Símbolos (TS)

A tabela de símbolos é uma estrutura essencial para armazenar informações sobre os identificadores e palavras reservadas encontrados no programa.

```
typedef struct {
    char lexema[50];
    char tipo[30];
} Simbolo;
```

```
Simbolo TS[1000];
```

```
int qtdTS = 0;
```

Descrição detalhada dos campos:

Campo	Tipo	Descrição
lexema	char[50]	Nome do identificador ou palavra reservada
tipo	char[30]	Categoria do símbolo (identificador ou palavra-reservada)
qtdTS	int	Quantidade de elementos atualmente armazenados

Características da implementação:

A tabela é implementada como um vetor estático de tamanho fixo (1000).

O controle de inserção é feito por meio da variável qtdTS.

A tabela é inicializada com todas as palavras reservadas da linguagem antes da análise.

A inserção de novos elementos verifica duplicatas usando strcmp.

A tabela é case-insensitive, ou seja, X, x e x são considerados o mesmo símbolo.

Apenas os seguintes elementos são armazenados: Identificadores, Palavras reservadas. Enquanto os que não são armazenados são Números, Operadores, Símbolos especiais

1.3 Variáveis globais de controle de posição

```
int linha = 1;
```

```
int coluna = 0;
```

A variável `coluna` é incrementada a cada caractere lido (`coluna++`). Ao encontrar um caractere de quebra de linha (`\n`), a `linha` é incrementada (`linha++`), e a `coluna` é reiniciada para zero (`coluna = 0`)

Essas variáveis permitem rastrear com precisão a localização de tokens e informar corretamente a posição de erros léxicos.

2. DESCRIÇÃO DAS FUNÇÕES IMPLEMENTADAS

2.1 Funções do arquivo `lexer.c`

```
int ehReservada(char *lex)
```

Função responsável por verificar se um lexema corresponde a uma palavra reservada da linguagem.

Possui um vetor estático contendo as 11 palavras reservadas:

```
char *reservadas[] = {
    "program", "var", "integer", "real", "begin", "end",
    "if", "then", "else", "while", "do"
};
```

Percorre o vetor comparando com o lexema recebido usando `strcasecmp` e retorna:

1 → se for palavra reservada

0 → caso contrário

Obs: A comparação é case-insensitive, garantindo compatibilidade com a linguagem.

```
void erro(char c)
```

Função responsável por registrar erros léxicos relacionados a caracteres inválidos.

Abre o arquivo `erros.err` em modo de adição ("a") e escreve a mensagem de erro contendo:

- caractere inválido
- linha
- coluna

Após isso, fecha o arquivo após escrita.

Formato da saída:

```
Erro: caractere invalido no arquivo de entrada '@' linha X
coluna Y
```

```
Token proximoToken(FILE *fp)
```

Função principal do analisador léxico. Implementa o AFD e retorna um token por chamada. Ela lê o arquivo caractere por caractere usando `fgetc` e Controla linha e coluna.

A função ignora espaços em branco, tabulações, quebras de linha e comentários.

Fluxo interno:

Entrada	Ação
Espaço, tab	Ignorado
\n	Incrementa linha e reinicia coluna
Letra (isalpha)	Inicia reconhecimento de identificador

Dígito (isdigit)	Inicia reconhecimento de número
{	Inicia leitura de comentário
<, >, :	Pode formar operadores compostos
+ - * / =	Operadores simples
; , () .	Símbolos
Outro	Erro léxico
EOF	Retorna token EOF

Reconhecimento de identificadores:

- Começa com letra
- Continua com letras ou dígitos
- Após leitura:
 - verifica se é palavra reservada
 - insere na TS

Reconhecimento de números:

- Sequência de dígitos → NUM_INT
- Caso encontre .:
 - verifica se há dígitos após
 - se sim → NUM_REAL
 - se não → erro léxico (número mal formado)

Comentários:

- Iniciam com {
- Consome caracteres até }
- Se atingir EOF sem fechar:
 - gera erro de comentário não fechado

Operadores compostos:

- $< \rightarrow <, <=, <>$
- $> \rightarrow >, >=$
- $: \rightarrow :, :=$

Uso de ungetc:

- Utilizado para devolver caracteres lidos indevidamente ao fluxo
- Importante para manter a correta leitura do próximo token

2.2 Funções do arquivo tabela_simbolos.c

```
void inserirTS(char *lexema, char *tipo)
```

Insere um símbolo na tabela, garantindo que não haja duplicação.

Percorre a tabela verificando se o lexema já existe (usando `strcasecmp`). Se não existir, copia lexema e tipo e incrementa `qtdTS`.

```
void salvarTS()
```

Grava o conteúdo da tabela de símbolos em arquivo. Percorre o vetor `TS[]` e escreve cada entrada no formato `lexema | tipo`

2.3 Funções do arquivo main.c

```
void inicializarTS()
```

Insere todas as palavras reservadas na tabela de símbolos antes da análise.

```
int main()
```

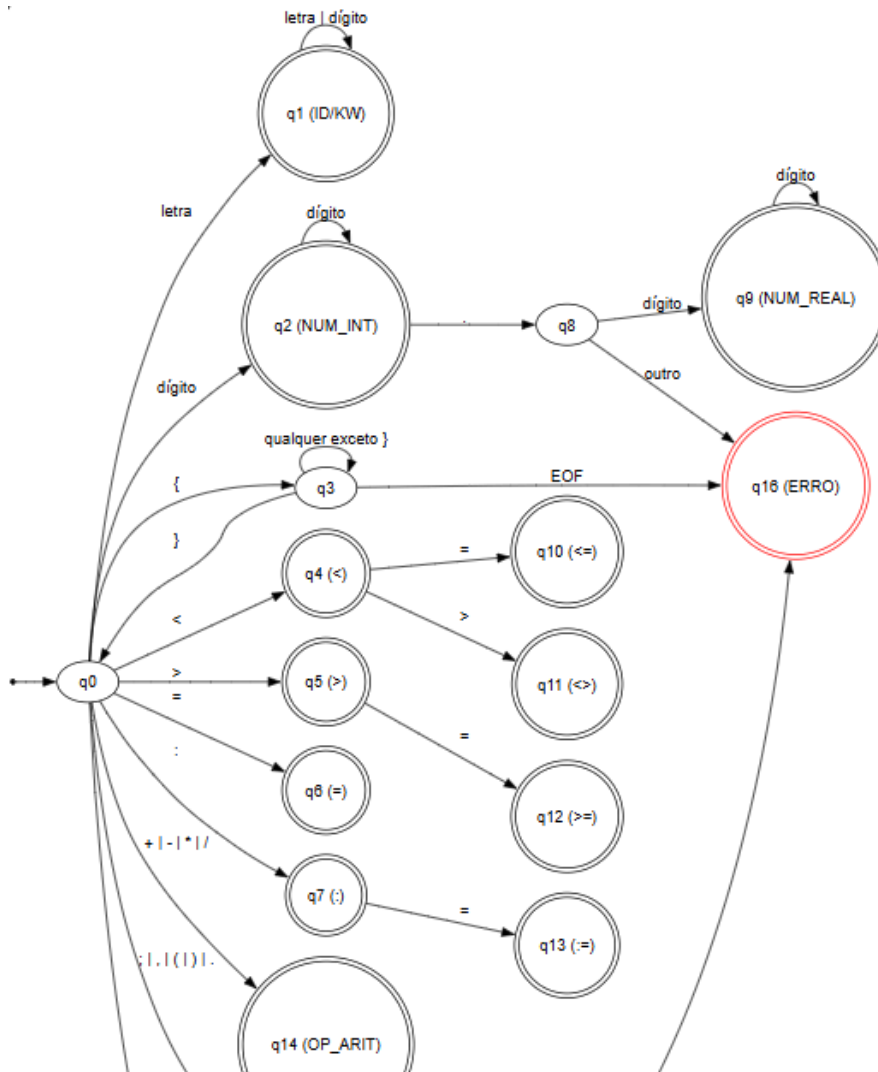
Função principal do programa, responsável por coordenar toda a execução.

Etapas:

1. Criação do diretório de saída (../saida)
2. Inicialização do arquivo de erros
3. Abertura dos arquivos:
 - entrada (entrada.pas)
 - saída (saida.lex)
4. Inicialização da TS
5. Loop de análise:
 - chama proximoToken() até EOF
 - escreve tokens no arquivo .lex
6. Salva a TS
7. Fecha arquivos

3. EXPLICAÇÃO DO AFD

O analisador léxico foi implementado com base em um Autômato Finito Determinístico (AFD), onde cada estado representa um tipo de reconhecimento léxico.



3.1 Estado inicial (q0)

Responsável por decidir o tipo de token com base no primeiro caractere:

- letra → identificador
- dígito → número
- { → comentário
- operadores → estados específicos
- símbolos → aceitação direta
- inválido → erro

3.2 Identificadores (q1)

- Reconhece sequências do tipo:

letra (letra | dígito)*

- Resultado:

- ID
- ou palavra reservada (verificação posterior)

3.3 Números

Estados:

- $q_2 \rightarrow$ número inteiro
- $q_8 \rightarrow$ ponto decimal
- $q_9 \rightarrow$ número real

Regra importante:

- A parte decimal é obrigatória após o ponto

Exemplo:

- $10 \rightarrow$ válido (inteiro)
- $3.14 \rightarrow$ válido (real)
- $20. \rightarrow$ erro

3.4 Comentários (q3)

- Início: {
- Fim: }
- Caso não feche $\rightarrow$ erro léxico
- Comentários são ignorados e não geram tokens

3.5 Operadores

Relacionais:

- $<, <=, <>$
- $>, >=$
- $=$

Atribuição:

- $:=, :$

Aritméticos:

- +, -, *, /

3.6 Símbolos

- ;, ,, (,), .

3.7 Estado de erro (q16)

Captura:

- caracteres inválidos (@, %, \$)
- números mal formados
- comentários não fechados

3.8 Transições completas

q0 → letra → q1 → ID/KW

q0 → dígito → q2 → NUM_INT

q2 → '.' → q8 → dígito → q9 → NUM_REAL

q8 → outro → ERRO

q0 → '{' → q3 → '}' → q0

q3 → EOF → ERRO

q0 → '<' → q4 → (= ou >)

q0 → '>' → q5 → (=)

q0 → ':' → q7 → (=)

q0 → operadores → q14

q0 → símbolos → q15

q0 → inválido → q16

Obs: Os caracteres { e } são utilizados exclusivamente para comentários e não geram tokens no analisador.

4. TESTES REALIZADOS

4.1 Testes corretos

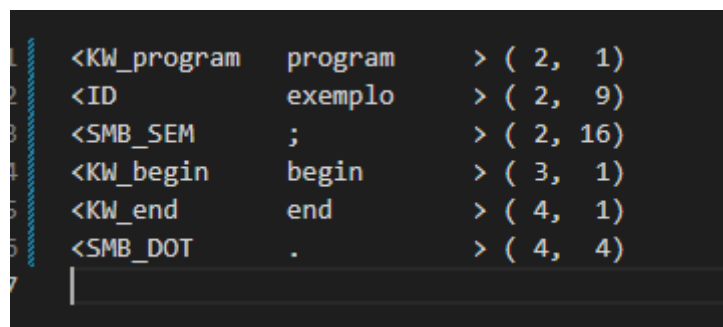
4.1.1 Teste 1

Entrada

```
program Exemplo;  
begin  
end.
```

Saída

<KW_program	program	> (2, 1)
<ID	exemplo	> (2, 9)
<SMB_SEM	;	> (2, 16)
<KW_begin	begin	> (3, 1)
<KW_end	end	> (4, 1)
<SMB_DOT	.	> (4, 4)



```
1 <KW_program  program  > ( 2, 1)  
2 <ID          exemplo  > ( 2, 9)  
3 <SMB_SEM     ;        > ( 2, 16)  
4 <KW_begin    begin    > ( 3, 1)  
5 <KW_end      end      > ( 4, 1)  
6 <SMB_DOT     .        > ( 4, 4)  
7 |
```

4.1.2 Teste 2

Entrada

```

program TesteVar;
var
x, y : integer;
z : real;
begin
end.

```

Saída

<KW_program	program	> (1, 1)
<ID	testevar	> (1, 9)
<SMB_SEM	;	> (1, 17)
<KW_var	var	> (2, 1)
<ID	x	> (3, 1)
<SMB_COM	,	> (3, 2)
<ID	y	> (3, 4)
<SMB_COL	:	> (3, 6)
<KW_integer	integer	> (3, 8)
<SMB_SEM	;	> (3, 15)
<ID	z	> (4, 1)
<SMB_COL	:	> (4, 3)
<KW_real	real	> (4, 5)
<SMB_SEM	;	> (4, 9)
<KW_begin	begin	> (5, 1)
<KW_end	end	> (6, 1)
<SMB_DOT	.	> (6, 4)

```

1  <KW_program   program   > ( 1, 1)
2  <ID           testevar  > ( 1, 9)
3  <SMB_SEM      ;         > ( 1, 17)
4  <KW_var       var       > ( 2, 1)
5  <ID           x         > ( 3, 1)
6  <SMB_COM      ,         > ( 3, 2)
7  <ID           y         > ( 3, 4)
8  <SMB_COL      :         > ( 3, 6)
9  <KW_integer   integer   > ( 3, 8)
10 <SMB_SEM      ;         > ( 3, 15)
11 <ID           z         > ( 4, 1)
12 <SMB_COL      :         > ( 4, 3)
13 <KW_real       real      > ( 4, 5)
14 <SMB_SEM      ;         > ( 4, 9)
15 <KW_begin      begin     > ( 5, 1)
16 <KW_end        end       > ( 6, 1)
17 <SMB_DOT      .         > ( 6, 4)
18 |

```

4.1.3 Teste 3

Entrada

```

program SistemaCalculo;

var

a, b, c : integer;

x, y : real;

begin

{ inicialização }

a := 10;

b := 20;

c := a + b * 3;

{ operações com real }

x := 2.5;

y := x * 10.0 + c;

```

end.

Saída

```

<KW_program    program    > ( 1,  1)
<ID            sistemacalculo> ( 1,  9)
<SMB_SEM      ;          > ( 1, 23)
<KW_var        var        > ( 2,  1)
<ID            a          > ( 3,  1)
<SMB_COM      ,          > ( 3,  2)
<ID            b          > ( 3,  4)
<SMB_COM      ,          > ( 3,  5)
<ID            c          > ( 3,  7)
<SMB_COL      :          > ( 3,  9)
<KW_integer    integer    > ( 3, 11)
<SMB_SEM      ;          > ( 3, 18)
<ID            x          > ( 4,  1)
<SMB_COM      ,          > ( 4,  2)
<ID            y          > ( 4,  4)
<SMB_COL      :          > ( 4,  6)
<KW_real       real       > ( 4,  8)
<SMB_SEM      ;          > ( 4, 12)
<KW_begin      begin      > ( 5,  1)
<ID            a          > ( 7,  1)
<OP_ASS       :=         > ( 7,  3)

```

<NUM_INT	10	> (7, 5)
<SMB_SEM	;	> (7, 7)
<ID	b	> (8, 1)
<OP_ASS	:=	> (8, 3)
<NUM_INT	20	> (8, 5)
<SMB_SEM	;	> (8, 7)
<ID	c	> (9, 1)
<OP_ASS	:=	> (9, 3)
<ID	a	> (9, 5)
<OP_AD	+	> (9, 7)
<ID	b	> (9, 9)
<OP_MUL	*	> (9, 11)
<NUM_INT	3	> (9, 13)
<SMB_SEM	;	> (9, 14)
<ID	x	> (11, 1)
<OP_ASS	:=	> (11, 3)
<NUM_REAL	2.5	> (11, 5)
<SMB_SEM	;	> (11, 8)
<SMB_SEM	;	> (11, 9)
<ID	y	> (12, 1)
<OP_ASS	:=	> (12, 3)
<ID	x	> (12, 5)
<OP_MUL	*	> (12, 7)
<NUM_REAL	10.0	> (12, 9)

<OP_AD	+	> (12, 15)
<ID	c	> (12, 17)
<SMB_SEM	;	> (12, 18)
<KW_end	end	> (13, 1)
<SMB_DOT	.	> (13, 4)

```

1  <KW_program  program  > ( 1, 1)
2  <ID          sistemacalculo> ( 1, 9)
3  <SMB_SEM    ;         > ( 1, 23)
4  <KW_var      var       > ( 2, 1)
5  <ID          a         > ( 3, 1)
6  <SMB_COM     ,         > ( 3, 2)
7  <ID          b         > ( 3, 4)
8  <SMB_COM     ,         > ( 3, 5)
9  <ID          c         > ( 3, 7)
10 <SMB_COL     :         > ( 3, 9)
11 <KW_integer  integer   > ( 3, 11)
12 <SMB_SEM    ;         > ( 3, 18)
13 <ID          x         > ( 4, 1)
14 <SMB_COM     ,         > ( 4, 2)
15 <ID          y         > ( 4, 4)
16 <SMB_COL     :         > ( 4, 6)
17 <KW_real     real      > ( 4, 8)
18 <SMB_SEM    ;         > ( 4, 12)
19 <KW_begin    begin     > ( 5, 1)
20 <ID          a         > ( 7, 1)
21 <OP_ASS      :=        > ( 7, 3)
22 <NUM_INT     10        > ( 7, 5)
23 <SMB_SEM    ;         > ( 7, 7)
24 <ID          b         > ( 8, 1)

```

4.2 Testes com erro léxico

4.2.1 Teste 1

Entrada

```

program ErroNumero;
begin
x := 20.;
end.

```

Erro

Erro: numero mal formado '20.' linha 4 coluna 5

Saída

<KW_program	program	> (2, 1)
<ID	erronumero	> (2, 9)
<SMB_SEM	;	> (2, 19)
<KW_begin	begin	> (3, 1)
<ID	x	> (4, 1)
<OP_ASS	:=	> (4, 3)
<ERRO	20.	> (4, 5)
<SMB_SEM	;	> (4, 6)
<KW_end	end	> (5, 1)
<SMB_DOT	.	> (5, 4)

```

1 <KW_program  program  > ( 2, 1)
2 <ID          erronumero > ( 2, 9)
3 <SMB_SEM     ;        > ( 2, 19)
4 <KW_begin    begin    > ( 3, 1)
5 <ID          x        > ( 4, 1)
6 <OP_ASS      :=       > ( 4, 3)
7 <ERRO        20.      > ( 4, 5)
8 <SMB_SEM     ;        > ( 4, 6)
9 <KW_end      end      > ( 5, 1)
10 <SMB_DOT     .        > ( 5, 4)

```

4.2.2 Teste 2

Entrada

```
program ErroChar;
begin
y := 10 @ 2;
end.
```

Erro

Erro: caractere invalido no arquivo de entrada '@'
linha 4 coluna 8

Saída

<KW_program	program	> (2, 1)
<ID	errochar	> (2, 9)
<SMB_SEM	;	> (2, 17)
<KW_begin	begin	> (3, 1)
<ID	y	> (4, 1)
<OP_ASS	:=	> (4, 3)
<NUM_INT	10	> (4, 5)
<NUM_INT	2	> (4, 10)
<SMB_SEM	;	> (4, 11)
<KW_end	end	> (5, 1)
<SMB_DOT	.	> (5, 4)

1	<KW_program	program	> (2, 1)
2	<ID	errochar	> (2, 9)
3	<SMB_SEM	;	> (2, 17)
4	<KW_begin	begin	> (3, 1)
5	<ID	y	> (4, 1)
6	<OP_ASS	:=	> (4, 3)
7	<NUM_INT	10	> (4, 5)
8	<NUM_INT	2	> (4, 10)
9	<SMB_SEM	;	> (4, 11)
10	<KW_end	end	> (5, 1)
11	<SMB_DOT	.	> (5, 4)
12			

4.2.3 Teste 3

Entrada

```

program ErroComentario;
begin
{ comentario nao fechado
x := 10;
end.

```

Erro

Erro: comentario nao fechado linha 6 coluna 4

Saída

```

<KW_program    program      > ( 2,  1)

<ID            errocomentario> ( 2,  9)

<SMB_SEM       ;            > ( 2, 23)

<KW_begin      begin        > ( 3,  1)

```

```

1  <KW_program  program    > ( 2,  1)
2  <ID          errocomentario> ( 2,  9)
3  <SMB_SEM     ;          > ( 2, 23)
4  <KW_begin    begin      > ( 3,  1)

```

5. SAÍDA GERADA PELO ANALISADOR (.lex)

Saída gerada pelo analisador do teste 4.1.3 :

```

1  <KW_program  program    > ( 1,  1)
2  <ID          sistemacalculo> ( 1,  9)
3  <SMB_SEM     ;          > ( 1, 23)
4  <KW_var      var        > ( 2,  1)
5  <ID          a          > ( 3,  1)
6  <SMB_COM     ,          > ( 3,  2)
7  <ID          b          > ( 3,  4)
8  <SMB_COM     ,          > ( 3,  5)
9  <ID          c          > ( 3,  7)
10 <SMB_COL     :          > ( 3,  9)
11 <KW_integer   integer    > ( 3, 11)
12 <SMB_SEM     ;          > ( 3, 18)
13 <ID          x          > ( 4,  1)
14 <SMB_COM     ,          > ( 4,  2)
15 <ID          y          > ( 4,  4)
16 <SMB_COL     :          > ( 4,  6)
17 <KW_real      real       > ( 4,  8)
18 <SMB_SEM     ;          > ( 4, 12)
19 <KW_begin     begin      > ( 5,  1)
20 <ID          a          > ( 7,  1)
21 <OP_ASS       :=         > ( 7,  3)
22 <NUM_INT      10         > ( 7,  5)
23 <SMB_SEM     ;          > ( 7,  7)
24 <ID          b          > ( 8,  1)

```

```
<KW_program  program    > ( 1,  1)
```

<ID	sistemacalculo>	(1, 9)
<SMB_SEM	;	> (1, 23)
<KW_var	var	> (2, 1)
<ID	a	> (3, 1)
<SMB_COM	,	> (3, 2)
<ID	b	> (3, 4)
<SMB_COM	,	> (3, 5)
<ID	c	> (3, 7)
<SMB_COL	:	> (3, 9)
<KW_integer	integer	> (3, 11)
<SMB_SEM	;	> (3, 18)
<ID	x	> (4, 1)
<SMB_COM	,	> (4, 2)
<ID	y	> (4, 4)
<SMB_COL	:	> (4, 6)
<KW_real	real	> (4, 8)
<SMB_SEM	;	> (4, 12)
<KW_begin	begin	> (5, 1)
<ID	a	> (7, 1)
<OP_ASS	:=	> (7, 3)
<NUM_INT	10	> (7, 5)
<SMB_SEM	;	> (7, 7)
<ID	b	> (8, 1)
<OP_ASS	:=	> (8, 3)

<NUM_INT	20	> (8, 5)
<SMB_SEM	;	> (8, 7)
<ID	c	> (9, 1)
<OP_ASS	:=	> (9, 3)
<ID	a	> (9, 5)
<OP_AD	+	> (9, 7)
<ID	b	> (9, 9)
<OP_MUL	*	> (9, 11)
<NUM_INT	3	> (9, 13)
<SMB_SEM	;	> (9, 14)
<ID	x	> (11, 1)
<OP_ASS	:=	> (11, 3)
<NUM_REAL	2.5	> (11, 5)
<SMB_SEM	;	> (11, 8)
<SMB_SEM	;	> (11, 9)
<ID	y	> (12, 1)
<OP_ASS	:=	> (12, 3)
<ID	x	> (12, 5)
<OP_MUL	*	> (12, 7)
<NUM_REAL	10.0	> (12, 9)
<OP_AD	+	> (12, 15)
<ID	c	> (12, 17)
<SMB_SEM	;	> (12, 18)
<KW_end	end	> (13, 1)

<SMB_DOT . > (13, 4)

6. TABELA DE SÍMBOLOS FINAL (.ts)

Tabela de símbolos do teste 4.1.3 :

1	program	palavra-reservada
2	var	palavra-reservada
3	integer	palavra-reservada
4	real	palavra-reservada
5	begin	palavra-reservada
6	end	palavra-reservada
7	if	palavra-reservada
8	then	palavra-reservada
9	else	palavra-reservada
10	while	palavra-reservada
11	do	palavra-reservada
12	sistemacalculo	identificador
13	a	identificador
14	b	identificador
15	c	identificador
16	x	identificador
17	y	identificador
18		

program | palavra-reservada

var | palavra-reservada

integer | palavra-reservada

real | palavra-reservada

begin | palavra-reservada

end | palavra-reservada

if | palavra-reservada

then | palavra-reservada

else	palavra-reservada
while	palavra-reservada
do	palavra-reservada
sistemacalculo	identificador
a	identificador
b	identificador
c	identificador
x	identificador
y	identificador